

Predicting Benchmark Scores from Sparse and Unpaired Cached Responses

Anonymous

Abstract

Evaluating a generative model on all queries from all available benchmark tasks is prohibitively expensive. As such, models are often evaluated on different subsets of tasks and/or different subsets of queries from a given task. The resulting collection of responses and the corresponding (model, task) score matrix are unpaired and sparse. In this paper, we introduce the Product Kernel Perspective Space (PKPS), a collection of low-dimensional representations of models based on their embedded responses. PKPS generalizes existing methods for representing black-box generative models by comparing responses to similar – rather than just identical – queries and is therefore able to utilize all available information. Theoretically, we show that benchmark prediction using PKPS is query-efficient relative to methods that only use information from dense and paired portions of the evaluation data. Empirically, we show that PKPS outperforms relevant baselines on two tasks across two large benchmark suites.

1 Introduction

Evaluating a generative model is necessary for understanding its capabilities. As the capabilities of generative models increase [Wei et al., 2022, Bubeck et al., 2023], their applicability expands to ever more domains—often necessitating the creation of new benchmarks for, e.g., mathematics [Hendrycks et al., 2021], law [Guha et al., 2023], finance [Islam et al., 2023], venture capital [Chen et al., 2025b], agentic tasks [Liu et al., 2024], and even running a vending machine business [Backlund and Petersson, 2025]. The current benchmarking landscape is correspondingly vast: BIG-bench alone contains more than 200 tasks [Srivastava et al., 2023], HELM tracks dozens of scenarios across multiple releases [Liang et al., 2023], and a recent compilation includes 133 different benchmarks for frontier models [Zeng and Papailiopoulos, 2026]. As such, a model is rarely evaluated on all benchmarks: of the (model, task) pairs in the compilation above, only 23% have a reported score [Zeng and Papailiopoulos, 2026]. Further, as benchmarks evolve – queries are added [Chollet et al., 2025], refreshed to avoid contamination [White et al., 2025], or subsampled to reduce cost [Perlitiz et al., 2024] – two models evaluated on the same benchmark may have responded to different sets of queries.

Benchmark prediction [Polo et al., 2024, Vivek et al., 2024, Li et al., 2024, Hofmann et al., 2025, Bowyer et al., 2026, Zeng and Papailiopoulos, 2026] attempts to alleviate the resource burden of evaluation by predicting scores from already-available information rather than computing them just from newly generated responses. There are two main benchmark prediction tasks: query-efficient evaluation and (model, task) score completion. In query-efficient evaluation, a model responds to a small subset of a task’s queries and its score on the full task is predicted from the resulting responses. Existing methods for query-efficient evaluation require access to item-level scores [Polo et al., 2024, Hofmann et al., 2025, Li et al., 2024, Bowyer et al., 2026] and/or cached embedded responses [Helm et al., 2026] to queries that all models have responded to and are typically not effective in settings where different models have been evaluated on different sets of queries. In (model, task) score completion, a model has not responded to any of a task’s queries and its score on the task is predicted from the observed entries of the score matrix. Existing methods for score completion use only the aggregate scores and discard the item-level responses that produced them [Zeng and Papailiopoulos, 2026]. In both tasks, the collection of available responses is often unpaired and sparse.

Table 1: MAE of benchmark prediction methods on both suites and tasks (best per column bolded, with ties both bolded; second best underlined; setup described in Section 4): query-efficiency at budgets $m \in \{1, 8\}$ queries per cell at full task coverage, and completion of missing cells at cohort sizes $n \in \{10, \max\}$ with task coverage p_{task} and query depth p_{query} fixed at 0.5, except for the sp. columns where $p_{\text{task}}=0.2$ and $n=10$. “—” denotes a setting where a method is not applicable.

Method	HELM suite (18 tasks, 93 models)					EEE suite (16 tasks, 45 models)				
	Query-eff.		Completion			Query-eff.		Completion		
	($m=1$)	($m=8$)	($n=10$)	($n=93$)	(sp.)	($m=1$)	($m=8$)	($n=10$)	($n=45$)	(sp.)
Sample score	0.292	<u>0.092</u>	—	—	—	0.224	0.081	—	—	—
IRT	0.348	0.343	—	—	—	0.165	0.139	—	—	—
LRMC	—	—	0.139	0.105	0.208	—	—	0.109	0.096	0.144
DKPS	0.168	0.109	—	—	—	0.116	0.077	—	—	—
PKPS	<u>0.136</u>	0.108	0.120	<u>0.103</u>	0.171	<u>0.076</u>	<u>0.058</u>	0.082	<u>0.070</u>	0.108
Ensemble	0.125	0.073	0.120	0.099	0.171	0.073	0.049	0.082	0.069	0.108

In this paper, we introduce the Product Kernel Perspective Space (PKPS) to predict benchmark scores directly from sparse, unpaired collections of responses. Our contributions can be summarized as follows:

1. *Conceptual*: we unify the two main benchmark prediction tasks as instances of a single problem: predicting scores from a sparse, unpaired collection of responses from previously evaluated models.
2. *Methodological & theoretical*: we propose the PKPS, a collection of low-dimensional representations of models defined on potentially unpaired embedded responses, and show that benchmark prediction using PKPS is at least as query-efficient as methods that only use information from paired responses (Theorem 2).
3. *Empirical*: we show that PKPS matches the accuracy of the sample score with up to an $8\times$ smaller query budget and, when ensembled with low-rank matrix completion, reduces score matrix completion error by up to 28%. Our empirical evaluation spans 2 large benchmark suites.

1.1 Related Work

Benchmark prediction. We decompose benchmark prediction into two lines of prior work: query-efficient benchmarking and (model, task) score completion. Methods for query-efficient benchmarking predict a model’s score on a full task from its responses to a small subset of the task’s queries—typically by selecting an informative query subset and extrapolating via IRT models [Polo et al., 2024, Hofmann et al., 2025], clustering [Vivek et al., 2024], rank-preserving subsampling [Perlitz et al., 2024], learned acquisition policies [Li et al., 2024], regression on item-level scores [Bowyer et al., 2026], or regression on model representations [Helm et al., 2026]. All previous methods proposed for query-efficient benchmarking implicitly require paired responses from previously evaluated models.

Methods for (model, task) score completion predict entries of the score matrix that were never evaluated, via cross-task predictability [Ye et al., 2023], observational scaling laws [Ruan et al., 2024], or low-rank matrix completion [Zeng and Papailiopoulos, 2026]. All operate exclusively on aggregate scores and discard information in the item-level scores or responses. Our proposed method uses item-level information from sparse and unpaired evaluation data to improve on methods from both literatures.

Low-dimensional representations of models. Our approach to sparse and unpaired benchmark prediction first constructs low-dimensional representations of the models before learning a simple regressor in the representation space. Existing approaches to represent models vary by access regime. For example, representations can be induced by comparing internal activations [Duderstadt

et al., 2023, Huh et al., 2024] and/or weights [Chen et al., 2025a] in the white-box setting. Evaluators do not necessarily have white-box access and instead rely on black-box methods such as the Data Kernel Perspective Space (DKPS) [Helm et al., 2024], which constructs low-dimensional representations by applying multidimensional scaling to distances between matrices of embedded responses on a paired set of queries. Recent theoretical work has established desirable theoretical properties such as consistency, concentration, and applicability to inference [Acharyya et al., 2024, 2025, Helm et al., 2025]. Most relevant to our setting, Helm et al. [2026] regress benchmark scores of previously evaluated models on DKPS representations and prove that the resulting predictions are query-efficient relative to the sample score. The proposed Product Kernel Perspective Space generalizes this construction with a kernel on the query space, enabling comparisons in the sparse and unpaired evaluation regime.

2 Benchmark Prediction

For our purposes, a model is a random mapping $f : \mathcal{Q} \rightarrow \mathcal{R}$ from a query space to a response space. A task is a pair $(Q^{(t)}, s^{(t)})$, where $Q^{(t)} \subseteq \mathcal{Q}$ is a collection of queries with $M^{(t)} := |Q^{(t)}|$ and $s^{(t)}$ is a scoring function mapping a model and a query collection to a score. Model i 's true score on task t is $y_i^{(t)} := s^{(t)}(f_i, Q^{(t)})$.

Evaluation data is indexed by (model, task, query) triples: for n models $f_1, \dots, f_n$ and T tasks $Q^{(1)}, \dots, Q^{(T)}$, the response of model i to query $q \in Q^{(t)}$ may or may not have been generated. Let $Q_i^{(t)} \subseteq Q^{(t)}$ be the queries for which model i 's response is observed, with $m_i^{(t)} := |Q_i^{(t)}|$ and $Q_i := \cup_t Q_i^{(t)}$. The corresponding *sample score* $\tilde{y}_i^{(t)} := s^{(t)}(f_i, Q_i^{(t)})$ is a noisy estimate of $y_i^{(t)}$.

Evaluation data is *sparse* when few models have $m_i^{(t)} > 0$ for a given task, and *unpaired* when the cross-model overlaps $m_{ii'}^{(t)} := |Q_i^{(t)} \cap Q_{i'}^{(t)}|$ are small. We let $\rho_{ii'}^{(t)} := m_{ii'}^{(t)} / m_i^{(t)}$ be the ratio of shared queries for a given model pair and note that $\rho_{ii'} \approx 1$ for all pairs in highly curated leaderboards and $\rho_{ii'} \approx 0$ in deployed evaluation settings. The goal of benchmark prediction is to estimate a model's score from the available evaluation data. The two tasks introduced in §1 are the two regimes of the observation count $m_i^{(t)}$:

1. *Query-efficient evaluation* is the case with $m_i^{(t)} > 0$. Responses or scores from other models on task t are typically available, and the goal is to outperform the naive sample score $\tilde{y}_i^{(t)}$.
2. *(Model, task) score matrix completion* is the case with $m_i^{(t)} = 0$. No responses from the target (model, task) slice exist and prediction must be based on model i 's responses to other tasks and on responses from other models evaluated on task t .

3 The Product Kernel Perspective Space

We estimate a model's score $y_i^{(t)}$ from the sparse, unpaired evaluation data of Section 2 by constructing low-dimensional representations of the models based on an appropriately defined pairwise distance matrix. We assume access to the responses for each observed (model, task, query) triple and to two embedding functions g_{query} and g_{response} that map queries and responses to $\mathbb{R}^{p_Q}$ and $\mathbb{R}^{p_R}$, respectively. Let $q_1, \dots, q_M$ be an enumeration of all queries and let x_{ij} denote the embedded response $f_i(q_j)$ and u_j denote the embedded query q_j .

Given a query kernel $k_Q : \mathbb{R}^{p_Q} \times \mathbb{R}^{p_Q} \rightarrow \mathbb{R}_{\geq 0}$ and a response kernel $k_R : \mathbb{R}^{p_R} \times \mathbb{R}^{p_R} \rightarrow \mathbb{R}_{\geq 0}$, we compare models i and i' via

$$A_{ii'} = \frac{\sum_{j \in Q_i} \sum_{l \in Q_{i'}} k_Q(u_j, u_l) k_R(x_{ij}, x_{i'l})}{\sum_{j \in Q_i} \sum_{l \in Q_{i'}} k_Q(u_j, u_l)}. \quad (1)$$

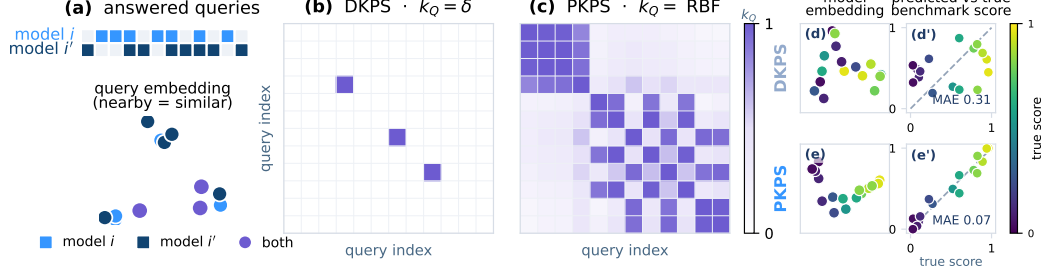


Figure 1: An illustration of the difference between the Product Kernel Perspective Space (PKPS) and the previously proposed Data Kernel Perspective Space (DKPS). **(a)** Top: two models respond to different sets of queries; bottom: queries represented as vectors. **(b)** The query kernel matrix used by DKPS: only information from queries both models have responded to is included in the dissimilarity matrix. **(c)** The query kernel matrix used by PKPS: information is weighted by query similarity in the embedding space. **(d–e’)** A toy benchmark-prediction example. Top: restricted to the rare shared queries, DKPS is noisy and correlates poorly with benchmark score (MAE 0.31). Bottom: PKPS leverages unpaired queries and is highly correlated with score (MAE 0.07).

Each entry is normalized by its own query-kernel mass, so models that answered different numbers of queries, or entirely different queries, remain comparable. We use the induced squared distance $D^2(i, i') = A_{ii} + A_{i'i'} - 2A_{ii'}$ to capture model (dis)similarity [Gretton et al., 2012]. Finally, the Product Kernel Perspective Space representations of the models— $\hat{\psi}_1, \dots, \hat{\psi}_n$ —are defined as the d -dimensional Euclidean configuration that minimizes the raw stress criterion to D [Trosset and Priebe, 2024]. That is,

$$(\hat{\psi}_1, \dots, \hat{\psi}_n) = \arg \min_{z_1, \dots, z_n \in \mathbb{R}^d} \sum_{i, i'=1}^n (\|z_i - z_{i'}\| - D(i, i'))^2. \quad (2)$$

We note that the definition of the PKPS strictly generalizes previous work by replacing the identity query kernel ($k_Q(u, u') = \mathbf{1}\{u = u'\}$) with a generic query kernel. Importantly, for any strictly positive k_Q , such as an RBF over the query embeddings, $D(i, i')$ includes information from unpaired responses and Eq. (1) is defined even on disjoint query sets. We highlight the difference between the previously proposed Data Kernel Perspective Space and the PKPS in Figure 1.

3.1 Theoretical properties

We next establish theoretical properties of the PKPS necessary for demonstrating its efficiency in sparse and unpaired benchmark prediction. In particular, the main result of this section shows that the representations concentrate around their population counterparts at a rate governed by the overlap of the models’ designs. We model the evaluation data of Section 2 as sampled (model, task, response) triples. Conditional on model i , each triple is an independent draw from its design distribution P_i : a task is sampled first, then a query conditioned on the sampled task, and finally the model’s response to the query. The population PKPS is defined by replacing the sums of Eq. (1) with expectations under the designs (Definition 2, Appendix E). Our analysis and implementation use the classical MDS solution: it coincides with the minimizer of Eq. (2) when D is Euclidean and is its standard tractable surrogate otherwise.

Our concentration results require bounded query and response kernels, bounded embedded responses, a positive cross-model query kernel mass, and a rank- d spectral gap in the population Gram matrix (see Assumptions 1–3, Appendix E). The required boundedness and the spectral gap are standard in spectral-embedding analyses [Yu et al., 2015, Acharyya et al., 2025]. The atypical requirement in our setting is the positive cross-model query kernel mass, though this holds trivially under a strictly positive k_Q and importantly does not assume that any individual query is shared. Let

$(\psi_1, \dots, \psi_n)$ denote the population PKPS, $\kappa = \inf_{i,i'} \mathbb{E}[k_Q(u, u')]$ the cross-model query kernel mass, and λ_d the d -th largest eigenvalue of the population Gram matrix.

Theorem 1 (Concentration of PKPS perspectives). *Suppose Assumptions 1–3 hold and that every model has responded to at least m queries, $\min_i |Q_i| \geq m$. Then for any $\delta \in (0, 1)$, with probability at least $1 - \delta$ there is an orthogonal W such that $\max_i \|\hat{\psi}_i - W \psi_i\| \leq C \kappa^{-2} \lambda_d^{-1/2} \sqrt{d} n \sqrt{\log(4n^2/\delta)}/2m$, where C depends only on the kernel bounds.*

Proof sketch. Each entry of Eq. (1) is a ratio of two-sample U-statistics. We thus get concentration of numerator and denominator via Hoeffding’s inequality [Hoeffding, 1963] and the bounded kernel assumption. The denominator is bounded below by κ and gives entrywise error $O(\kappa^{-2} m^{-1/2})$ (Lemma 1). A Weyl–Procrustes perturbation argument [Tu et al., 2016, Lemma 5.4] then transfers the entrywise error to the rank- d spectral embedding at the stated rate (Lemma 2). The full proof is in Appendix E. $\square$

The rate of PKPS concentration is the rate of paired DKPS concentration scaled by the population overlap (κ^{-2}), and does not require paired queries.

3.2 Inference in the PKPS

We next describe benchmark prediction in the PKPS and establish its query efficiency. Inference using the PKPS is a two-step statistical problem: first embed the models, then train a decision function that uses the representations as proxies for the models [Helm et al., 2026].

Fix a task t and let $f, f_1, \dots, f_n$ be i.i.d. draws from a model distribution, observed through the sampled (model, task, response) triples of Section 3.1. Every observed quantity is a function of the triples: the embedded responses enter Eq. (1), and the sample-score vector $(\hat{y}_i^{(1)}, \dots, \hat{y}_i^{(T)})$ collects per-task statistics of model i ’s responses. The true score $y_i := s^{(t)}(f_i, Q^{(t)})$ is a functional of the model and is never observed. The models with $m_i^{(t)} > 0$ serve as *references* for task t , and the *target* is the model whose true score we estimate—its own cell may be observed but noisy (query-efficient evaluation) or missing (score completion). We embed all $n+1$ models jointly via Eq. (2), treat $\hat{\psi}_i$ as a proxy for f_i , fit a regressor $\hat{h} : \mathbb{R}^d \rightarrow \mathcal{Y}$ on the observed pairs $\{(\hat{\psi}_i, \hat{y}_i^{(t)})\}_{i=1}^n$, and predict $\hat{y}(f) = \hat{h}(\hat{\psi})$. We note that the procedure is naturally transductive—Eq. (1) uses no scores, so the target is embedded alongside the references—though out-of-sample extensions exist [Trosset and Priebe, 2008].

The main result of this section shows that benchmark prediction with the PKPS is query-efficient relative to paired DKPS. Beyond Assumptions 1–3, our result requires a bounded and Lipschitz score on the population perspective space—the working premise of any response-based predictor (Figure 1e)—and a model distribution that places mass on every neighborhood of it, so that the target has near reference neighbors (Assumptions 4 and 5, Appendix E). In the following, MSE denotes mean squared error over the draw of the target model and its triples, m denotes a per-model query budget, $\min_i |Q_i| \geq m$, and σ_q^2 denotes the per-query score variance.

Definition 1 (Query efficiency; following Helm et al., 2026). Fix a per-model query budget m and let h_n denote a score estimator using n scored reference models. h is *query-efficient relative to h'* if there exists N such that $\text{MSE}(h_n) \leq \text{MSE}(h'_n)$ with high probability for all $n \geq N$.

Theorem 2 (Query efficiency relative to paired DKPS). *Fix a per-model budget m and pairing fraction $\rho \in [0, 1]$. Let the scoring functions be means of bounded per-query scores, let each observed cell be a uniform subsample at budget m , and write PKPS σ for the estimator with fixed bandwidth σ and a k -nearest-neighbor regressor fit on the sample scores $\hat{y}_i^{(t)}$. Under Assumptions 1–5, there exists a bandwidth σ such that PKPS σ is query-efficient relative to paired DKPS. Under a fully unpaired design ($\rho = 0$), the separation is strict: every paired-DKPS estimator has $\text{MSE} \geq \text{Var}(y)$*

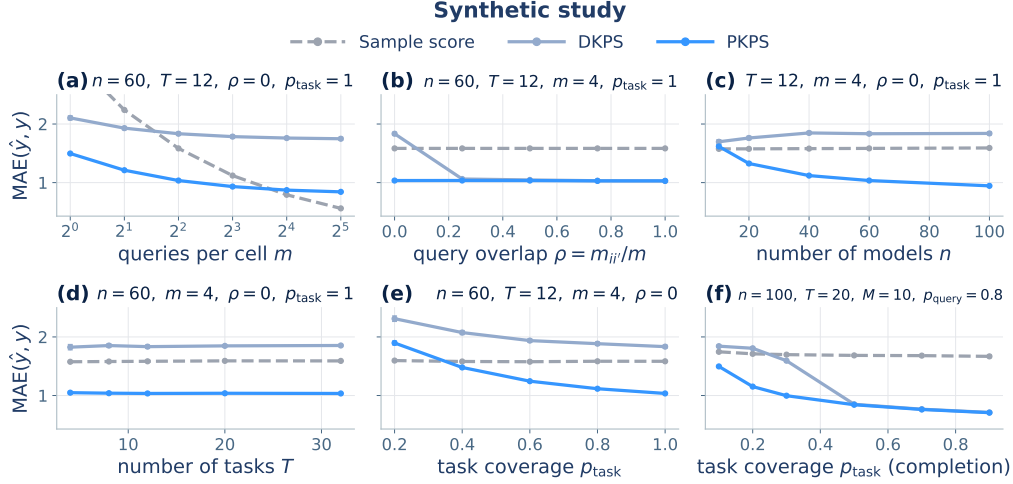


Figure 2: Score MAE on the synthetic benchmark (± 1 SEM over 50 seeds; DKPS light blue, PKPS bright blue, sample score dashed gray; fixed parameters in each panel title). **(a–e)** Query-efficient evaluation: every observed cell is measured with m noisy queries and we predict its true score, as a function of the budget m (a), the cross-model overlap ρ (b), the cohort size n (c), the number of tasks T (d), and the task coverage p_{task} (e), at unpaired designs ($\rho=0$) except where ρ is swept. **(f)** Score completion of held-out cells as a function of task coverage; the gray line is the task mean of observed scores (no sample score exists for a held-out cell).

at every (n, m) , while for any $\varepsilon > 0$ there exist (n, m, k) , with m polynomial in $(1/\varepsilon, 1/\kappa, n)$ and independent of ρ , at which $\text{MSE}(\text{PKPS}_\sigma) \leq \varepsilon$ with high probability.

Proof sketch. The exact-matching limit coincides with paired DKPS on the shared queries with the same regression head, which gives the first claim with equality. With exact reference scores, Theorem 1 and Assumption 5 supply a bandwidth attaining any ε with $m = \text{poly}(n, 1/\kappa, 1/\varepsilon)$, and no step involves ρ (Lemma 3). The reference noises $\tilde{y}_i^{(t)} - y_i$ are mean zero, bounded, of variance at most σ_q^2/m , and independent across models under unpaired designs, so the k -neighbor average contributes at most $\sigma_q^2/(mk)$, removed by letting k grow. With no shared queries, every identity-matched comparison is an empty sum, so a paired-DKPS prediction cannot depend on the target’s responses and its error is at least the score variance. The full proofs are in Appendix E. $\square$

Helm et al. [2026] established that paired DKPS is query-efficient relative to the sample score. The comparison extends to PKPS at every pairing fraction.

Corollary 1 (Relative to the sample score). *The sample score’s MSE at budget m is σ_q^2/m for every n . By the achievability part of Theorem 2, PKPS is query-efficient relative to the sample score whenever $\varepsilon < \sigma_q^2/m$, at every pairing fraction.*

In practice, the bandwidth is selected by cross-validation over a finite grid containing the exact-matching limit (Section 4, Appendix E). Empirically, a bandwidth that bridges similar queries improves on exact matching at every $\rho < 1$ (Figure 2e).

The same machinery covers (model, task) score completion ($m_i^{(t)} = 0$): since Eq. (1) uses only responses, a model that has answered none of task t ’s queries is embedded from its responses to other tasks, and the regression transfers the scores of reference models evaluated on t . Identity-matched constructions have no mechanism for this regime and the sample score does not exist. We evaluate completion empirically in Section 4.

4 Experiments

We evaluate PKPS on synthetic data and on two large benchmark suites.

Synthetic data. We generate n models with latent ability vectors $v_i \in \mathbb{R}^5$ and T tasks with latent vectors $t_k \in \mathbb{R}^5$, scores $v_i \cdot t_k + \varepsilon$, and queries drawn near each task center ($u_j = t_k + \eta_j$ with isotropic noise η_j ; a model’s embedded response to u_j is centered at $u_j + v_i$). Each model answers m queries per task. We control unpairedness via $p = m_{i'}/m$ and sparsity via a probability of observing a task p_{task} and, conditioned on observing a task, a probability of answering a query p_{query} .

Benchmark suites. We consider two large benchmark suites. The first consists of 18 tasks from HELM-Lite [Liang et al., 2023]. The tasks have very different response types – MATH (Hendrycks et al., 2021, 7 math subjects), WMT-14 (Bojar et al., 2014, 5 translation directions), MedQA (Jin et al., 2021, medical multiple-choice), and LegalBench (Guha et al., 2023, 5 legal classification subsets). We consider 93 models that have been evaluated on all 18 tasks. As collected, the suite is dense and fully paired ($\rho_{i'j'} = 1$): we induce sparsity and unpairedness by subsampling each model’s observed queries.

The second suite is built from the Every Eval Ever datastore [EvalEval Coalition, 2026], a community-maintained collection of item-level evaluation results covering 36 benchmarks and 165 models and growing. The store exhibits the sparsity and unpairedness that motivate this paper: only 15% of (model, benchmark) pairs have been evaluated and 59% of model pairs share no benchmark at all. Within a benchmark, pairedness is bimodal: on 26 of 33 multi-model benchmarks, models answered essentially identical item sets (mean $\rho_{i'j'} > 0.99$), while the remaining seven – all arena-style [Chiang et al., 2024] – are severely unpaired (mean $\rho_{i'j'}$ between 0.27 and 0.66). We study a dense submatrix of the store so that true scores are available and sparsity and pairedness can be controlled in our evaluations: 16 tasks (MATH-MC levels 1–5 [Hendrycks et al., 2021], GSM-MC [Cobbe et al., 2021], GPQA-Diamond [Rein et al., 2024], and nine judging categories from JudgeBench [Tan et al., 2025] and RewardBench 2 [Malik et al., 2025]) and the 45 models that answered every item of every task. We randomly sample 32 responses for each model for each task and treat the full benchmark score as the target.

Both suites were chosen because they include item-level responses – most large benchmark leaderboards only include overall or subtask scores. Details describing the HELM and EEE data such as the included models, included tasks, the number of queries per task, etc. are provided in Appendices A and B, respectively.

Methods and baselines. We evaluate PKPS on both query-efficiency and (model, task) score prediction. Unless otherwise stated we embed all responses and queries into a 3072-dimensional space via gemini-embedding-001 [Lee et al., 2025], apply PCA on the collection of embedded queries and embedded responses, and use an RBF kernel on the reduced query space and a linear kernel on the reduced response space. Given the PKPS pairwise distance matrix, we embed the models in a $d = 8$ space and use k -nearest neighbor regression with $k = 5$.

Depending on the task we include different baselines. Each produces an estimate of $y_i^{(t)}$:

1. *Query-efficiency.* We compare PKPS to the sample score (i.e., $\hat{y}_i^{(t)}$), paired DKPS [Helm et al., 2026], Item Response Theory (“IRT”, Polo et al., 2024) and a convex combination of PKPS and sample score (i.e., $\alpha \hat{y}_i^{(t)} + (1 - \alpha) \hat{y}_{i,\text{PKPS}}^{(t)}$).

2. *Score completion.* We compare PKPS to Low-Rank Matrix Completion (“LRMC”, Zeng and Papailiopoulos, 2026) and a convex combination of PKPS and LRMC (i.e., $\alpha \hat{y}_{i,\text{LRMC}}^{(t)} + (1 - \alpha) \hat{y}_{i,\text{PKPS}}^{(t)}$).

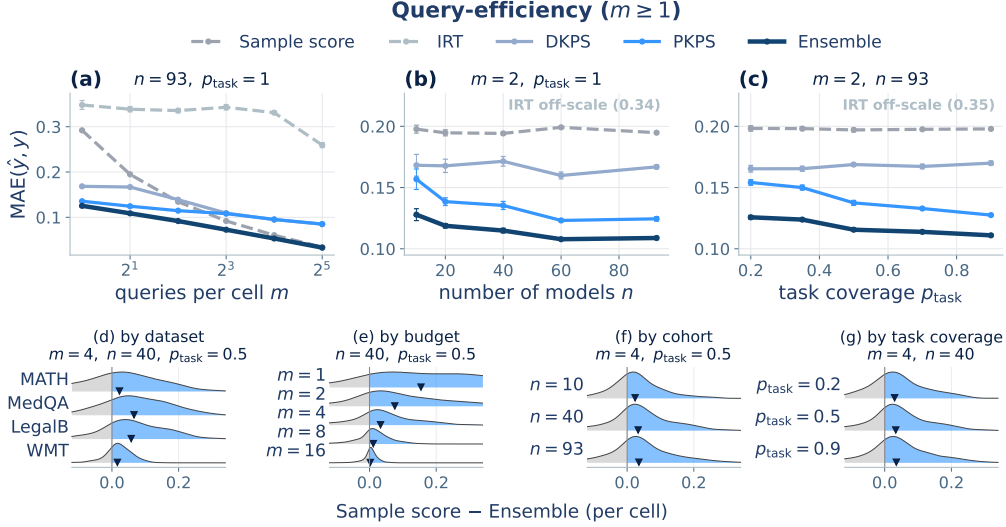


Figure 3: Query-efficient evaluation on the HELM suite. (a–c) MAE against the true full-evaluation score when each model answers m unpaired queries per observed cell (± 1 SEM), as a function of budget m , cohort size n , and task coverage p_{task} . IRT is poorly identified without a shared item block and is omitted from (b, c); its value is annotated. (d–g) Per-cell difference $\Delta = \text{Sample score} - \text{Ensemble MAE}$ (blue = Ensemble better) by dataset, budget, cohort, and coverage, with the other levers fixed at their sweep median; ▼ marks the per-row mean.

Both the query kernel RBF bandwidth and the convex coefficient for the ensemble are selected via leave-one-out cross validation on the observed data. We report average mean absolute error (MAE) of the predicted score versus the true score for every setting over 16 seeds (50 on synthetic data). Additional experimental details, such as the sweep grids, PCA dimensions, and cross-validation protocols, are provided in Appendix A.

4.1 Results

PKPS is query-efficient on the synthetic data. Figure 2 shows the MAE for PKPS, DKPS, and the sample score under the query-efficient evaluation protocol: every observed cell carries a budget of m queries and we predict its true score. The panels vary (a) the query budget, (b) the pairwise query overlap, (c) the number of models, (d) the number of tasks, and (e) the task coverage. Panel (f) studies score completion, where the targets are held-out cells and the score-only baseline is the task mean.

PKPS outperforms DKPS everywhere and the sample score at small query budgets (a). Panel (b) is the empirical counterpart of the strict separation in Theorem 2. At $\rho = 0$, DKPS is worse than the sample score, while a single shared query ($\rho = 0.25$ at $m = 4$) restores it to PKPS’s level. PKPS itself is flat in ρ as it is unaffected by the sampled overlap since it only requires distributional overlap (Assumption 2). As n grows for a fixed sparsity and pairedness, PKPS can take advantage of additional reference models while DKPS cannot (c). Task coverage behaves similarly (e) and the number of tasks matters little for either method (d). In the completion panel (f), PKPS outperforms DKPS when observed cells are scarce and they are matched for moderate to large coverage.

PKPS-based methods outperform the sample score at small query budgets. Figure 3a–c shows the MAE for PKPS, DKPS, IRT, the sample score, and the ensemble on the HELM suite: each model answers its own independently sampled queries – the real-data analogue of $\rho = 0$ – and we predict its true full-evaluation score. At $m=1$, PKPS attains 0.136 and the ensemble 0.125 versus 0.292 for the sample score, so PKPS with one query matches the sample score at four (a). As the budget

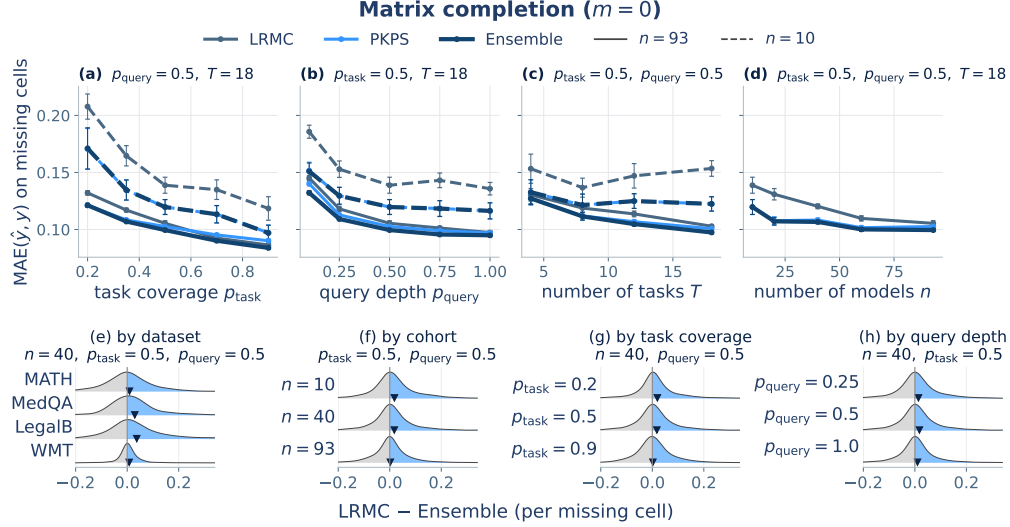


Figure 4: Score completion on the HELM suite. (a–d) MAE on missing cells (± 1 SEM) when a fraction p_{task} of (model, task) cells is observed at query depth p_{query} , as a function of task coverage, query depth, number of tasks, and cohort size (LRM slate, PKPS bright blue, Ensemble navy; solid $n=93$, dashed $n=10$). (e–h) Per-missing-cell difference $\Delta = \text{LRM} - \text{Ensemble}$ MAE (blue = Ensemble better) by dataset, cohort, coverage, and query depth, with the other levers fixed at their sweep median; ▼ marks the per-row mean.

grows, the sample score eventually overtakes the representations’ residual bias, and the ensemble follows the better of its two components throughout (a). PKPS improves steadily with the number of reference models ($0.157 \rightarrow 0.124$, b) and with the task coverage ($0.154 \rightarrow 0.128$, c), while DKPS, which cannot use the additional unpaired responses, is flat. IRT is poorly identified without a shared item block and is worse than the sample score at every budget.

The query-efficiency gains are broad but tail-driven. Figure 3d–g shows the distribution of the per-cell difference between the sample score’s and the ensemble’s absolute errors, by dataset (d), budget (e), cohort (f), and task coverage (g). The ensemble outperforms the sample score on most cells and in the mean of every row. The gains are largest on the rich-response datasets (d) and at the smallest budgets (e), where a single response carries the most information, and they shrink toward zero as the budget grows. The mean gain is positive at every cohort size (f) and every coverage (g).

The same representations complete missing cells. Figure 4a–d shows MAE on missing cells when a fraction p_{task} of cells is observed, each at depth p_{query} . The sample score does not exist for a missing cell, so the comparison is against LRM. LRM requires many rows to estimate its factors and degrades when models are scarce: at $n=10$ its MAE is 0.139 versus 0.120 for PKPS (Figure 4, dashed). At the full cohort ($n=93$, solid), PKPS slightly edges LRM (0.103 vs. 0.105) and the ensemble performs best (0.099).

The completion gains are modest but consistent. Figure 4e–h shows the per-missing-cell difference between LRM’s and the ensemble’s absolute errors, by dataset, cohort, coverage, and query depth. The win is smaller than in the query-efficiency task but stays positive in the mean across every lever: the response channel adds information wherever the score matrix alone is rank-deficient, and never subtracts it.

The results are qualitatively similar on the Every Eval Ever suite. The second suite shows the same qualitative behavior with a larger completion margin (lever sweeps in Appendix B). At $m=1$, PKPS attains a third of the sample score’s error (0.076 vs. 0.224) and matches the accuracy the sample score reaches at $m=8$ (0.081): an $8\times$ smaller budget. On missing cells, the ensemble outperforms LRMC at every operating point, including the full cohort (0.069 vs. 0.096 at $n=45$). Table 1 collects both tasks on both suites, and the ensemble is best or tied in every column.

The results are insensitive to representation and pipeline choices. Varying the MDS dimension over 4–24 moves PKPS’s MAE by less than 0.005, and the cross-validated bandwidth matches the best fixed choice (Appendix C). Ablations of the regression head, the response kernel, the ensemble weighting rule, and the incidental query overlap in the EEE pools are in Appendix D.

5 Discussion

In this paper we introduced the PKPS, which compares models through their responses to similar, rather than identical, queries. Outside of curated leaderboards, evaluation data is typically unpaired: each model answers the queries its evaluators chose, and methods that require a shared block of identical queries do not apply. Our central finding is that benchmark prediction remains possible in this regime: as $\rho \rightarrow 0$, DKPS and IRT degrade sharply while PKPS does not. The same representations support both tasks we studied—query-efficient evaluation and (model, task) score completion—and the ensemble performs at least as well as either component alone.

Limitations and future work. We close with the current limitations and the extensions they suggest. First, PKPS embeds every observed response and query with a single fixed embedder. We did not ablate this choice, and embedding functions are known to have a nonnegligible effect at small budgets [Helm et al., 2026]. Second, the block-diagonal instantiation on the HELM suite disables bridging across response domains. A shared response space that lets responses in one domain inform another remains untested. Third, computing Eq. (1) is quadratic in the number of queries per model pair. Fourth, the achievability part of Theorem 2 is existential and provides no rate in n , and strict domination at intermediate ρ rests on some bandwidth improving on exact matching, which we establish empirically rather than by a lower bound. Finally, the per-cell gains over score-only baselines are modest on dense low-rank matrices and decisive in the sparse, noisy, unpaired regime that motivates the problem.

Reproducibility statement

All methods are released as a documented Python package (the PKPS estimator, matrix completion, and IRT baselines), with the data-preparation, experiment, and figure scripts for both the synthetic study and the HELM suite; every experiment is seeded, all real-data numbers are means over 16 seeds (synthetic: 50), and hyperparameters are selected by the leak-free cross-validation described in Section 4 (leakage is checked by an automated test that corrupts one model’s held-out scores and verifies its predictions are unchanged). Appendix A lists the suite composition, embedding models, and all hyperparameter settings.

References

Aranyak Acharyya, Michael W. Trosset, Carey E. Priebe, and Hayden S. Helm. Consistent estimation of generative model representations in the Data Kernel Perspective Space. *arXiv preprint arXiv:2409.17308*, 2024.

- Aranyak Acharyya, Joshua Agterberg, Youngser Park, and Carey E. Priebe. Concentration bounds on response-based vector embeddings of black-box generative models. *arXiv preprint arXiv:2511.08307*, 2025.
- Axel Backlund and Lukas Petersson. Vending-bench: A benchmark for long-term coherence of autonomous agents. *arXiv preprint arXiv:2502.15840*, 2025.
- Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Lebeling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, Radu Soricut, Lucia Specia, and Aleš Tamchyna. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*, 2014.
- Sam Bowyer, Acyr Locatelli, and Kris Cao. Efficient benchmarking is just feature selection and multiple regression. *arXiv preprint arXiv:2605.25773*, 2026.
- Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, Marco Tulio Ribeiro, and Yi Zhang. Sparks of artificial general intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712*, 2023.
- Nan Chen, Hayden Helm, Youngser Park, Carey Priebe, and Soledad Villar. Extracting information from fine-tuned weights. In *Non-Euclidean Foundation Models: Advancing AI Beyond Euclidean Frameworks*, 2025a.
- Rick Chen, Joseph Ternasky, Afriyie Samuel Kwesi, Ben Griffin, Aaron Ontoyin Yin, Zakari Salifu, Kelvin Amoaba, Xianling Mu, Fuat Alican, and Yigit Ihlamur. VCBench: Benchmarking LLMs in venture capital. *arXiv preprint arXiv:2509.14448*, 2025b.
- Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios N. Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot Arena: An open platform for evaluating LLMs by human preference. In *International Conference on Machine Learning (ICML)*, 2024.
- François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Brandon Duderstadt, Hayden S. Helm, and Carey E. Priebe. Comparing foundation models using data kernels. *arXiv preprint arXiv:2305.05126*, 2023.
- EvalEval Coalition. Every Eval Ever: A unified open data format and public datastore for AI evaluation results. <https://evalevalai.com/projects/every-eval-ever/>, 2026. Dataset: https://huggingface.co/datasets/evaleval/EEE_datastore.
- Arthur Gretton, Karsten M. Borgwardt, Malte J. Rasch, Bernhard Schölkopf, and Alexander Smola. A kernel two-sample test. *Journal of Machine Learning Research*, 13:723–773, 2012.
- Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, Adam Zambrano, et al. Legalbench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2023.
- Hayden Helm, Brandon Duderstadt, Youngser Park, and Carey E. Priebe. Tracking the perspectives of interacting language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.

- Hayden Helm, Aranyak Acharyya, Youngser Park, Brandon Duderstadt, and Carey Priebe. Statistical inference on black-box generative models in the Data Kernel Perspective Space. In *Findings of the Association for Computational Linguistics (ACL)*, 2025.
- Hayden Helm, Ben Johnson, and Carey E. Priebe. Query-efficient model evaluation using cached responses. *arXiv preprint arXiv:2605.07096*, 2026.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.
- Valentin Hofmann, David Heineman, Ian Magnusson, Kyle Lo, Jesse Dodge, Maarten Sap, Pang Wei Koh, Chun Wang, Hannaneh Hajishirzi, and Noah A. Smith. Fluid language model benchmarking. In *Conference on Language Modeling*, 2025.
- Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The Platonic Representation Hypothesis. In *International Conference on Machine Learning (ICML)*, 2024.
- Pranab Islam, Anand Kannappan, Douwe Kiela, Rebecca Qian, Nino Scherrer, and Bertie Vidgen. Financebench: A new benchmark for financial question answering. *arXiv preprint arXiv:2311.11944*, 2023.
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14), 2021.
- Jinhyuk Lee, Zhuyun Dai, Sai Meher Karthik Duddu, and et al. Gemini embedding: Generalizable embeddings from Gemini. *arXiv preprint arXiv:2503.07891*, 2025.
- Yang Li, Jie Ma, Miguel Ballesteros, Yassine Benajiba, and Graham Horwood. Active evaluation acquisition for efficient LLM benchmarking. *arXiv preprint arXiv:2410.05952*, 2024.
- Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, Benjamin Newman, Binhang Yuan, Bobby Yan, Ce Zhang, Christian Cosgrove, Christopher D. Manning, Christopher Ré, Diana Acosta-Navas, Drew A. Hudson, Eric Zelikman, Esin Durmus, Faisal Ladhak, Frieda Rong, Hongyu Ren, Huaxiu Yao, Jue Wang, Keshav Santhanam, Laurel Orr, Lucia Zheng, Mert Yuksekgonul, Mirac Suzgun, Nathan Kim, Neel Guha, Niladri Chatterji, Omar Khattab, Peter Henderson, Qian Huang, Ryan Chi, Sang Michael Xie, Shibani Santurkar, Surya Ganguli, Tatsunori Hashimoto, Thomas Icard, Tianyi Zhang, Vishrav Chaudhary, William Wang, Xuechen Li, Yifan Mai, Yuhui Zhang, and Yuta Koreeda. Holistic evaluation of language models. *Transactions on Machine Learning Research (TMLR)*, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.
- Saumya Malik, Valentina Pyatkin, Sander Land, Jacob Morrison, Noah A. Smith, Hannaneh Hajishirzi, and Nathan Lambert. RewardBench 2: Advancing reward model evaluation. *arXiv preprint arXiv:2506.01937*, 2025.
- Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking (of language models). In *North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

- Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *International Conference on Machine Learning (ICML)*, 2024.
- David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A graduate-level google-proof Q&A benchmark. In *First Conference on Language Modeling*, 2024.
- Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *Advances in Neural Information Processing Systems*, 2024.
- Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research*, 2023.
- Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. In *International Conference on Learning Representations*, 2025.
- Michael W. Trosset and Carey E. Priebe. The out-of-sample problem for classical multidimensional scaling. *Computational Statistics & Data Analysis*, 52(10):4635–4642, 2008.
- Michael W. Trosset and Carey E. Priebe. Continuous multidimensional scaling. *arXiv preprint arXiv:2402.04436*, 2024.
- Stephen Tu, Ross Boczar, Max Simchowitz, Mahdi Soltanolkotabi, and Benjamin Recht. Low-rank solutions of linear matrix equations via Procrustes flow. In *International Conference on Machine Learning (ICML)*, 2016.
- Rajan Vivek, Kawin Ethayarajh, Diyi Yang, and Douwe Kiela. Anchor points: Benchmarking models with much fewer examples. In *Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2024.
- Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Transactions on Machine Learning Research*, 2022.
- Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddhartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. LiveBench: A challenging, contamination-free LLM benchmark. In *International Conference on Learning Representations (ICLR)*, 2025.
- Qinyuan Ye, Harvey Yiyun Fu, Xiang Ren, and Robin Jia. How predictable are large language model capabilities? a case study on BIG-bench. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.
- Yi Yu, Tengyao Wang, and Richard J. Samworth. A useful variant of the Davis–Kahan theorem for statisticians. *Biometrika*, 102(2):315–323, 2015.
- Yuchen Zeng and Dimitris Papailiopoulos. You don’t need to run every eval. *arXiv preprint arXiv:2606.24020*, 2026.

A Experimental details

Suite. 18 tasks over four datasets from HELM [Liang et al., 2023]: MATH (7 subjects), WMT-14 (5 translation directions), MedQA, and LegalBench (5 subsets), on the 93 models evaluated on all four; Table 2 lists every task with its query count (the loader subsamples each task to at most 120 queries with a fixed seed, so datasets contribute comparably). MATH and WMT-14 responses are free text and are embedded with gemini-embedding-001 (3072-d); MedQA and LegalBench responses are single-token answers, encoded one-hot. Queries share a single gemini-embedding-001 embedding space. Each dataset’s responses occupy a disjoint block: with a linear k_R , cross-dataset response similarity is exactly zero, and a within-domain query bandwidth keeps k_Q near zero across datasets, so the product kernel factorizes by dataset. Response and query embeddings are reduced by PCA fit on observed rows only, with dimension chosen by the second Zhu–Ghodsli elbow.

Table 2: HELM suite composition: 18 tasks over four datasets, with the number of unique queries per task. Every task is scored for all 93 shared models; the loader caps each task at 120 queries (seeded subsample).

Dataset	Task	Queries	Dataset	Task	Queries
MATH	algebra	135	WMT-14	cs-en	873
MATH	counting & probability	39	WMT-14	de-en	776
MATH	geometry	38	WMT-14	fr-en	754
MATH	intermediate algebra	52	WMT-14	hi-en	387
MATH	number theory	30	WMT-14	ru-en	613
MATH	prealgebra	86	MedQA	med.qa	1000
MATH	precalculus	57	LegalBench	abercrombie	95
LegalBench	corporate lobbying	490	LegalBench	citizenship questions	1000
LegalBench	function of decision section	367	LegalBench	proa	95

Model coverage (HELM). The 93 shared models span 20 developers: 01-ai-yi-34b, 01-ai-yi-6b, 01-ai-yi-large-preview, AlephAlpha.luminous-base, AlephAlpha.luminous-extended, AlephAlpha.luminous-supreme, ai21-j2-grande, ai21-j2-jumbo, ai21-jamba-1.5-large, ai21-jamba-1.5-mini, ai21-jamba-instruct, allenai-olmo-7b, amazon-nova-lite-v1, amazon-nova-micro-v1, amazon-nova-pro-v1, anthropic-claude-2.0, anthropic-claude-2.1, anthropic-claude-3-5-haiku-20241022, anthropic-claude-3-5-sonnet-20240620, anthropic-claude-3-5-sonnet-20241022, anthropic-claude-3-haiku-20240307, anthropic-claude-3-opus-20240229, anthropic-claude-3-sonnet-20240229, anthropic-claude-instant-1.2, anthropic-claude-instant-v1, anthropic-claude-v1.3, cohere-command, cohere-command-light, cohere-command-r, cohere-command-r-plus, databricks-dbrx-instruct, deepseek-ai-deepseek-llm-67b-chat, deepseek-ai-deepseek-v3, google-gemini-1.0-pro-001, google-gemini-1.0-pro-002, google-gemini-1.5-flash-001, google-gemini-1.5-flash-002, google-gemini-1.5-pro-001, google-gemini-1.5-pro-002, google-gemini-1.5-pro-preview-0409, google-gemini-2.0-flash-exp, google-gemma-2-27b-it, google-gemma-2-9b-it, google-gemma-7b, google-text-bison@001, google-text-unicorn@001, meta-llama-2-13b, meta-llama-2-70b, meta-llama-2-7b, meta-llama-3-70b, meta-llama-3-8b, meta-llama-3.1-405b-instruct-turbo, meta-llama-3.1-70b-instruct-turbo, meta-llama-3.1-8b-instruct-turbo, meta-llama-3.2-11b-vision-instruct-turbo, meta-llama-3.2-90b-vision-instruct-turbo, meta-llama-3.3-70b-instruct-turbo, meta-llama-65b, microsoft-phi-2, microsoft-phi-3-medium-4k-instruct, microsoft-phi-3-small-8k-instruct, mistralai-mistral-7b-instruct-v0.3, mistralai-mistral-7b-v0.1, mistralai-mistral-large-2407, mistralai-mistral-medium-2312, mistralai-mixtral-8x22b, mistralai-mixtral-8x7b-32kseqlen, mistralai-open-mistral-nemo-2407, nvidia-nemotron-4-340b-instruct, openai-gpt-3.5-turbo-0613, openai-gpt-4-0613, openai-gpt-4-1106-preview, openai-gpt-4-turbo-2024-04-09, openai-gpt-4o-2024-05-13, openai-gpt-4o-2024-08-06, openai-gpt-4o-mini-2024-07-18, openai-text-davinci-002, openai-text-davinci-003, qwen.qwen1.5-110b-chat, qwen.qwen1.5-14b, qwen.qwen1.5-32b, qwen.qwen1.5-72b, qwen.qwen1.5-7b, qwen.qwen2-72b-instruct, qwen.qwen2.5-72b-instruct-turbo, qwen.qwen2.5-7b-instruct-turbo, snowflake-snowflake-arctic-instruct, tiuae-falcon-40b, tiuae-falcon-7b, upstage-solar-pro-241126, writer-palmyra-x-004, writer-palmyra-x-v2, writer-palmyra-x-v3.

PKPS. RBF query kernel with bandwidth selected per run by leave-one-model-out k -NN ($k=8$) on the observed scores over the grid $\sigma \in \{0.03, 0.1, 0.3, 1, 3\} \times \text{median pairwise query distance}$,

plus the near-delta (DKPS) limit; linear response kernel on real data, RBF on synthetic. Response blocks are capped at 48 PCA components per dataset and the shared query space at 64; each (model, query) pair uses one sampled response ($r=1$). Models are embedded by classical MDS with $d=8$ throughout, following Helm et al. [2026] and fixed a priori (d = latent dimension on synthetic). Appendix C shows the results are insensitive to d over 4–24, and that the cross-validated bandwidth tracks the best fixed choice.

Prediction head. All embedding methods share one head: logit-transformed scores, per-task standardization to unit variance, a two-way (model + task) additive bias, and a distance-weighted k -NN ($k=5$) regression of the residual on the model embedding, fit *leave-one-family-out*: all models from the target’s developer are excluded from its training set, so predictions never lean on near-duplicate siblings [Helm et al., 2026]. Held-out cells are predicted directly; observed cells leave their own target out. The head is identical to the LRMC baseline’s, so the two methods differ only in how the residual is modeled (k -NN on the perspective versus a low-rank factorization) and performance differences are attributable to the representation. Matching the per-task standardization is important: without it, the model offset is dominated by the tasks with the largest logit variance.

Baselines. Low-rank matrix completion (LRMC) follows BenchPress [Zeng and Papailiopoulos, 2026]: logit space, per-task standardization, two-way bias, rank-2 ALS with ridge $\lambda=0.1$, averaged over 5 random initializations; on observed cells it is cross-fit so a cell’s prediction never sees its own sample. IRT is a 1PL (Rasch) model on binary tasks: item difficulties by maximum likelihood (L-BFGS) on the largest fully-observed (model $\times$ item) block, per-model ability by bounded scalar MLE, clipped to $[-4, 4]$. The sample baseline is the cell’s own m -query mean.

Ensemble. On observed cells, the prediction is $\alpha \hat{y}_{\text{sample}} + (1 - \alpha) \hat{y}_{\text{PKPS}}$ with a scalar α per held-out family, selected on a 21-point grid by leave-one-family-out validation against the other families’ cached full scores (the same information the regressor trains on); on missing cells, PKPS and LRMC are blended with a weight fit by held-out cross-validation against observed scores. No quantity ever uses the scores of the family being predicted. Appendix D compares this rule to a closed-form inverse-variance weighting and to the fixed m/M schedule of Helm et al. [2026].

Sweeps and reporting. All real-data sweeps use 16 seeds; curves and table entries report the per-seed mean over tasks, then the mean (± 1 SEM) over seeds. Query efficiency: budgets $m \in \{1, 2, 4, 8, 16, 32\}$ at full cohort and coverage; cohort $n \in \{10, 20, 40, 60, 93\}$ (HELM) or $\{10, 20, 30, 45\}$ (EEE) and task coverage $p_{\text{task}} \in \{0.2, 0.35, 0.5, 0.7, 0.9\}$, both at the few-query point $m=2$. Completion: task coverage as above and query depth $p_{\text{query}} \in \{0.1, 0.25, 0.5, 0.75, 1.0\}$ with cohort lines $n \in \{10, \text{full}\}$; task count $T \in \{4, 8, 12, 18\}$ (HELM) or $\{4, 8, 12, 16\}$ (EEE); the cohort sweep holds $p_{\text{task}}=p_{\text{query}}=0.5$.

Synthetic study. 50 seeds per configuration; latent dimension 5, observed embedding dimension 20, response noise 0.5; both kernels RBF. Query-efficiency panels (Figure 2a–e): every observed cell carries m queries and a sample score with measurement noise $4/\sqrt{m}$; the true score is predicted by a distance-weighted average of the $k=8$ nearest models’ sample scores, restricted to cells they observe. Completion panel (f): score noise 0.1, held-out cells predicted by k -NN regression with $k=5$. Unswept levers default to $n=60$, $T=12$, $m=4$, $p=0$, $p_{\text{task}}=1$; panel-specific settings are given in each figure title.

Compute. All experiments run on a single multi-core CPU node; text embeddings are precomputed once through the embedding API and cached. Embedding the EEE suite (30,304 pooled

responses plus 7,665 queries, $\sim 11\text{M}$ tokens) took ~ 13 minutes and under \$2 of API credit; each 16-seed sweep completes in minutes (EEE) to hours (HELM) on the same node.

B The Every Eval Ever suite

The second suite is built from the Every Eval Ever datastore [EvalEval Coalition, 2026], an MIT-licensed, community-maintained collection of item-level evaluation results in a unified format (inputs, raw model outputs, and per-item scores). The full store covers 36 benchmarks and 165 models with only 15% of (model, benchmark) pairs evaluated (median 5 benchmarks per model), and 59% of model pairs share no benchmark. Within a benchmark, 26 of the 33 with at least two models are essentially fully paired (23 with identical item sets across every model pair), while the seven arena-style benchmarks (six Fibble arenas and Wordle arena) are natively unpaired: mean per-pair overlap 0.27–0.66, median zero on six of the seven, and per-model coverage of the item union 0.15–0.33—in interactive evaluation, the queries a model sees depend on its matchups. We take the five benchmarks with the widest item-level model coverage—MATH-MC, GSM-MC, GPQA-Diamond, JudgeBench, and RewardBench 2—restricted to the 45 models present on all five, for which every model answered every item; sub-tasks (MATH-MC levels 1–5; four JudgeBench and five RewardBench 2 categories) give 16 tasks (Table 3). Every domain is free text (including the judging benchmarks’ rationales), so all responses and queries are embedded with `gemini-embedding-001` and blocked by benchmark exactly as in Appendix A. Each cell’s observable pool is 32 queries drawn with a model-dependent seed, so two models share only incidental queries (median $\rho_{ii'}^{(t)} = 0.09$, mean 0.18, largest on the smallest tasks); the query-efficiency budget m and the completion depth p_{query} subsample this pool, and the prediction target is always the cell’s full-depth mean over all items, which is never embedded. All scores are binary, so IRT applies to every task (on the HELM suite it excludes WMT). Figures 5 and 6 repeat the lever sweeps of Figures 3 and 4 on this suite; Table 1 summarizes.

Table 3: EEE suite composition: 16 tasks over five benchmarks, with the number of items per (model, task) cell. Every task is fully answered by all 45 shared models; each cell’s observable pool is 32 items, sampled per model.

Benchmark	Task	Items	Benchmark	Task	Items
MATH-MC	level 1	430	JudgeBench	coding	42
MATH-MC	level 2	882	JudgeBench	knowledge	154
MATH-MC	level 3	1115	JudgeBench	math	56
MATH-MC	level 4	1199	JudgeBench	reasoning	98
MATH-MC	level 5	1288	RewardBench 2	factuality	475
GSM-MC	gsm.mc	1319	RewardBench 2	focus	495
GPQA-Diamond	gpqa.diamond	198	RewardBench 2	math	183
RewardBench 2	precise if	160	RewardBench 2	safety	450

Model coverage (EEE). The 45 models shared by all five benchmarks span 13 developers:

`cohere/c4ai-command-a-03-2025`, `cohere/c4ai-command-r-08-2024`,
`cohere/c4ai-command-r-plus-08-2024`, `cohere/c4ai-command-r7b-12-2024`,
`cohere/command-a-reasoning-08-2025`, `deepseek/deepseek-r1-0528`,
`deepseek/deepseek-v3-1-terminus`, `deepseek/deepseek-v3-2`, `deepseek/deepseek-v3-2-speciale`,
`deepseek/deepseek-v4-flash-fp8`, `deepseek/deepseek-v4-pro`, `google/gemini-3-1-pro-preview`,
`google/gemma-2-27b-it`, `google/gemma-2-9b-it`, `google/gemma-3-12b-it`, `google/gemma-3-27b-it`,
`google/gemma-4-e2b-it`, `google/gemma-4-e4b-it`, `llm360/k2-v2-instruct`,
`meta/llama-4-maverick-17b-128e-instruct-fp8`, `meta/meta-llama-3-1-8b-instruct`,
`minimax/minimax-m2-5`, `mistral/mistral-small-4-119b-2603`, `moonshot/kimi-k2-5`,
`moonshot/kimi-k2-6`, `moonshot/kimi-k2-thinking`, `openai/gpt-5-5`, `openai/gpt-5-mini`,
`openai/gpt-5-nano`, `openai/gpt-oss-120b`, `openai/gpt-oss-20b`, `qwen/qwen3-30b-a3b-thinking-2507`,
`qwen/qwen3-5-122b-a10b`, `qwen/qwen3-5-2b`, `qwen/qwen3-5-397b-a17b`, `qwen/qwen3-6-35b-a3b`,
`qwen/qwen3-6-plus`, `qwen/qwen3-next-80b-a3b-thinking`, `stepfun/step-3-5-flash`,
`xiaomi/mimo-v2-flash`, `zai-org/glm-4-6-fp8`, `zai-org/glm-4-7-flash`, `zai-org/glm-4-7-fp8`,
`zai-org/glm-5-1-fp8`, `zai-org/glm-5-fp8`.

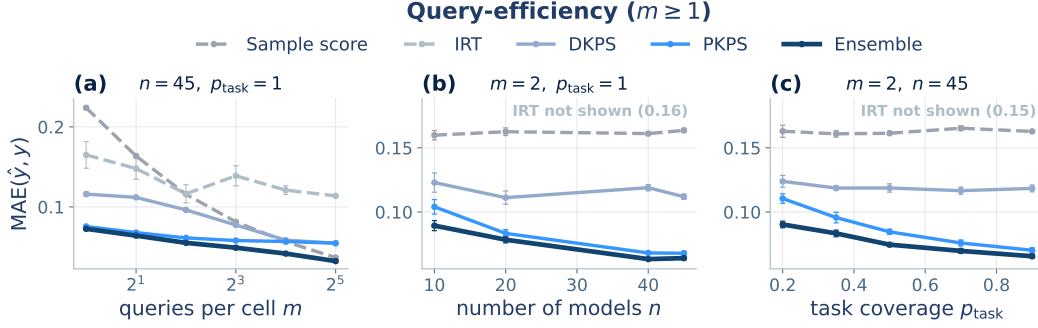


Figure 5: Query-efficient evaluation on the Every Eval Ever suite (45 models, 16 tasks; MAE against the true full-evaluation score, ± 1 SEM over 16 seeds). The protocol matches Figure 3. PKPS and the Ensemble are the most accurate at every budget; DKPS trails because the query pools are unpaired, and IRT is poorly identified.

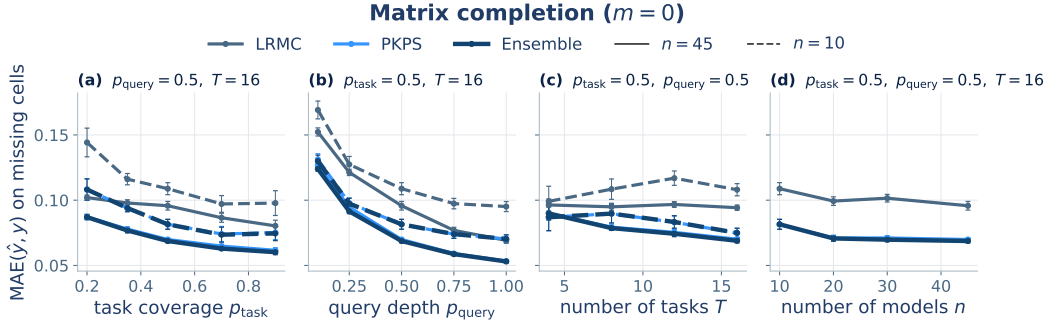


Figure 6: Score completion on the Every Eval Ever suite (MAE on missing cells, ± 1 SEM; solid $n=45$, dashed $n=10$). The protocol matches Figure 4. Unlike on the HELM suite, PKPS and the Ensemble outperform LRMC at every operating point.

C Sensitivity analyses

Figure 7 probes the two representation hyperparameters on the query-efficiency experiment (16 seeds throughout). **MDS dimension (a, b).** Repeating the budget sweep at $d \in \{4, 8, 24\}$ moves PKPS’s MAE by less than 0.005 at every budget on both suites (at the smallest budgets, smaller d is very slightly better—fewer coordinates to estimate from one noisy query per cell); the paper’s choice ($d=8$, following Helm et al., 2026) is never the worst point at any budget. **Query bandwidth (c).** Fixing the bandwidth at each grid multiplier instead of cross-validating traces the expected curve: near-delta multipliers and the exact-matching limit ($k_Q(u, u') = \mathbf{1}\{u = u'\}$, abbreviated δ) are substantially worse (unpaired pools leave them little to compare), broad multipliers are best, and the leak-free cross-validated choice matches the best fixed multiplier on both suites (0.126 vs. 0.125 on HELM, 0.069 vs. 0.068 on EEE)—the one hyperparameter that matters is the one the method selects automatically. Sensitivity to the number of reference models is part of the main experiments (the cohort sweeps of Figures 3b and 4d).

D Pipeline ablations

Each ablation repeats the query-efficiency budget sweep (16 seeds, both suites) with one pipeline choice changed; all other settings match Appendix A.

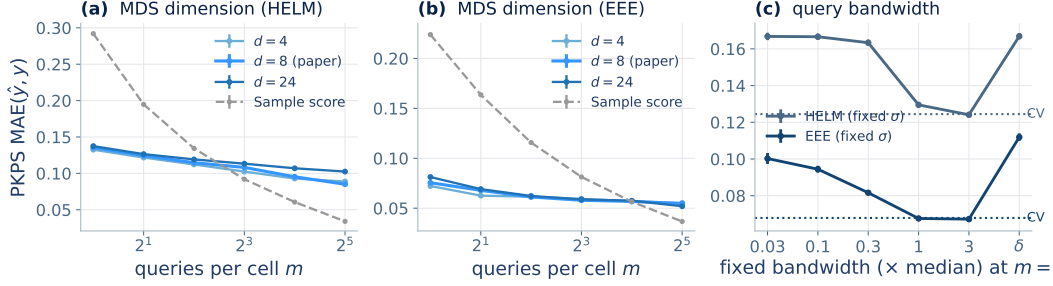


Figure 7: Sensitivity analyses on the query-efficiency experiment (± 1 SEM, 16 seeds). (a, b) PKPS MAE against budget at MDS dimension $d \in \{4, 8, 24\}$ (paper choice in red; sample dashed); the curves are nearly indistinguishable on both suites. (c) MAE at $m=2$ with the query bandwidth fixed at each grid multiplier ($\delta = \text{exact matching}$); dotted lines mark the cross-validated selection, which matches the best fixed choice on both suites.

Ensemble weighting. We compare the paper’s cross-validated scalar weight against two alternatives: a closed-form inverse-variance rule (the sample score weighted by $e_{PKPS}^2 / (e_{PKPS}^2 + \tilde{p}(1 - \tilde{p})/m)$ per cell, with $\tilde{p}$ the PKPS estimate and e_{PKPS}^2 the depth-weighted, noise-debiased disagreement with the observed samples) and the fixed schedule $\alpha = m/M$ of Helm et al. [2026]. The cross-validated weight is best at every budget on both suites: on HELM it improves on the inverse-variance rule from 0.132 to 0.125 at $m=1$ and from 0.077 to 0.073 at $m=8$; on EEE from 0.084 to 0.073 and from 0.052 to 0.049. The m/M schedule trails both on HELM (0.096 at $m=8$) and is close to the cross-validated weight on EEE, where the small pools ($M=32$) let m/M reach 1.

Regression head. Replacing the distance-weighted k -NN head with ordinary least squares helps slightly on HELM (0.132 vs. 0.136 at $m=1$; 0.080 vs. 0.085 at $m=32$), consistent with Helm et al. [2026], but hurts on EEE, increasingly so at large budgets (0.066 vs. 0.055 at $m=32$). We use k -NN as the robust default across suites.

Response kernel. An RBF response kernel in place of the linear one changes PKPS’s MAE by at most 0.003 at every budget on both suites; the linear choice is justified by the block-diagonal factorization (Appendix A) at no accuracy cost.

Incidental overlap. The EEE pools share $\sim 10\%$ of items across models by chance. Enforcing exactly disjoint query sets (true $\rho = 0$) changes PKPS by at most 0.005 at $m \leq 8$ (0.080 vs. 0.076 at $m=1$), and IRT becomes inapplicable outright—no shared item block exists. At $m \geq 16$ the disjointness constraint exhausts the 32-item pools and mechanically shortens the sample score’s effective depth, so comparisons there reflect the constraint rather than the methods.

E Proofs of Theorems 1 and 2 and Corollary 1

Formal definitions and assumptions. We collect the formal statements referenced in Section 3.

Definition 2 (Population PKPS). Let each model i ’s (task, query, response) triples be drawn i.i.d. from a design distribution P_i . The *population comparison* is $A_{ii'}^* = \mathbb{E}[k_Q(u, u') k_R(x_i, x_{i'})] / \mathbb{E}[k_Q(u, u')]$ with draws from P_i and $P_{i'}$ independent, the *population squared distances* are $D^{*2}(i, i') = A_{ii}^* + A_{i'i'}^* - 2A_{ii'}^*$, and the *population perspectives* $\psi_1, \dots, \psi_n \in \mathbb{R}^d$ are the rank- d spectral embedding of the centered Gram matrix B^* of D^{*2} .

Assumption 1 (Boundedness). Kernels and embedded responses are bounded.

Assumption 2 (Population overlap). The *cross-model query-kernel mass* is bounded below: $\kappa = \inf_{i,i'} \mathbb{E}[k_Q(u, u')] > 0$.

Assumption 3 (Spectral identifiability). B^* has rank d with spectral gap $\lambda_d > 0$.

Assumption 4 (Score regularity). The benchmark score is bounded and γ -Lipschitz on the population perspective space.

Assumption 5 (Model support). The model distribution places mass on every neighborhood of the population perspective space.

Population quantities. Each of model i 's triples independently draws a task, then a query conditioned on the task, then the model's response (Section 3.1); write $z = (u, x_i(u))$ for an embedded query–response pair (response noise included) and P_i for its law. By Assumption 1 the kernels are bounded: $k_Q \in [0, 1]$ after normalization (RBF) and $|k_R| \leq B_R$. Define the population comparison

$$A_{ii'}^* = \frac{\mathbb{E}[k_Q(u, u') k_R(x_i(u), x_{i'}(u'))]}{\mathbb{E}[k_Q(u, u')]}, \quad z \sim P_i, z' \sim P_{i'} \text{ independent},$$

population squared distances $D^{*2}(i, i') = A_{ii}^* + A_{i'i'}^* - 2A_{ii'}^*$, the centered Gram matrix $B^* = -\frac{1}{2}JD^{*2}J$ with $J = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$, and population perspectives $\Psi^* \in \mathbb{R}^{n \times d}$ its rank- d spectral embedding, which exists with $\lambda_d(B^*) > 0$ by Assumption 3 (cf. Definition 2).

Lemma 1 (Affinity concentration). *Under Assumptions 1 and 2, for models i, i' with $\min(|Q_i|, |Q_{i'}|) \geq m$ and any $\delta \in (0, 1)$: with probability at least $1 - \delta$,*

$$|A_{ii'} - A_{ii'}^*| \leq \frac{2(1 + B_R)}{\kappa^2} t_m(\delta), \quad t_m(\delta) = \bar{B} \sqrt{\frac{\log(4/\delta)}{2m}}, \quad \bar{B} = \max(2B_R, 1),$$

provided $t_m(\delta) \leq \kappa/2$.

Proof. The numerator $N = \frac{1}{|Q_i||Q_{i'}|} \sum_{j,l} k_Q(u_j, u'_l) k_R(x_{ij}, x_{i'l})$ and denominator $Z = \frac{1}{|Q_i||Q_{i'}|} \sum_{j,l} k_Q(u_j, u'_l)$ of Eq. (1) are two-sample U -statistics of order $(1, 1)$ in the i.i.d. pairs $z_j \sim P_i, z'_l \sim P_{i'}$, with kernels of range at most $2B_R$ and 1 respectively (response noise enters only through the bounded kernel argument, so the number of responses per query may stay fixed). By Hoeffding's inequality for two-sample U -statistics [Hoeffding, 1963, Section 5], each deviates from its mean by more than its range times $\sqrt{\log(4/\delta)/2m}$ with probability at most $\delta/2$; since $\bar{B}$ dominates both ranges, a union bound gives $|N - N^*| \leq t_m(\delta)$ and $|Z - Z^*| \leq t_m(\delta)$ jointly with probability $\geq 1 - \delta$. On this event, since $Z^* \geq \kappa$ by Assumption 2 and $t_m(\delta) \leq \kappa/2$, we have $Z \geq \kappa/2$, and

$$|A_{ii'} - A_{ii'}^*| = \frac{|NZ^* - N^*Z|}{ZZ^*} \leq \frac{|N - N^*|}{Z} + \frac{|N^*||Z - Z^*|}{ZZ^*} \leq \frac{2t_m(\delta)}{\kappa} + \frac{2B_R t_m(\delta)}{\kappa^2} \leq \frac{2(1 + B_R)}{\kappa^2} t_m(\delta),$$

using $|N^*| \leq B_R$ and $\kappa \leq 1$. □

Lemma 2 (Perspective perturbation). *Let $\varepsilon_\infty = \max_{i,i'} |A_{ii'} - A_{ii'}^*|$. There is an orthogonal $W \in \mathbb{R}^{d \times d}$ such that*

$$\max_i \|\hat{\psi}_i - W\psi_i\| \leq \frac{25\sqrt{d} n \varepsilon_\infty}{\sqrt{\lambda_d(B^*)}}.$$

Proof. Entrywise, $|D^2 - D^{*2}| \leq 4\varepsilon_\infty$, and double centering inflates an entrywise bound by at most a factor of 4, so $E = B - B^*$ satisfies $\|E\|_{\text{op}} \leq \|E\|_F \leq 8n\varepsilon_\infty$. Let B_d be the top- d truncation of B that classical MDS embeds ($B_d = \hat{\Psi}\hat{\Psi}^\top$, positive semidefinite). By Weyl's inequality every eigenvalue of

B beyond the d -th, and every negative eigenvalue, has magnitude at most $\|E\|_{\text{op}}$ (since $\lambda_{d+1}(B^*) = 0$ and $B^* \succeq 0$), so $\|B_d - B\|_{\text{op}} \leq \|E\|_{\text{op}}$. The matrix $B_d - B^*$ has rank at most $2d$, hence

$$\|B_d - B^*\|_F \leq \sqrt{2d} \|B_d - B^*\|_{\text{op}} \leq \sqrt{2d} (\|B_d - B\|_{\text{op}} + \|E\|_{\text{op}}) \leq 2\sqrt{2d} \|E\|_F.$$

Both $B_d = \hat{\Psi}\hat{\Psi}^\top$ and $B^* = \Psi^*\Psi^{*\top}$ are positive semidefinite of rank at most d , so by the Procrustes alignment lemma of [Tu et al. \[2016, Lemma 5.4\]](#),

$$\min_{W \in O(d)} \|\hat{\Psi} - \Psi^*W\|_F^2 \leq \frac{\|B_d - B^*\|_F^2}{2(\sqrt{2} - 1)\lambda_d(B^*)}.$$

Combining, $\max_i \|\hat{\psi}_i - W\psi_i\| \leq \|\hat{\Psi} - \Psi^*W\|_F \leq \frac{2\sqrt{2d}}{\sqrt{2}(\sqrt{2}-1)} \cdot \frac{8n\epsilon_\infty}{\sqrt{\lambda_d(B^*)}} \leq \frac{25\sqrt{d}n\epsilon_\infty}{\sqrt{\lambda_d(B^*)}}$. Nearest-neighbor distances are invariant to W . $\square$

Lemma 3 (Achievability with exact reference scores). *Suppose the reference scores are observed exactly ($\hat{y}_i^{(t)} = y_i$). Under Assumptions 1–5, for any $\epsilon > 0$ there exist a bandwidth σ and (n, m) , with m polynomial in $(1/\epsilon, 1/\kappa, n)$ and independent of ρ , at which $\text{MSE}(\text{PKPS}_\sigma) \leq \epsilon$ with high probability.*

Proof of Lemma 3. Take scores in $[0, 1]$ without loss of generality (Assumption 4). Fix $\epsilon > 0$ and set targets $c \leq \sqrt{\epsilon/2}/(8\gamma)$ for the estimation error and $\epsilon' \leq \sqrt{\epsilon/2}/(2\gamma)$ for the population nearest-neighbor distance. By Assumption 5 there is $n_0(\epsilon')$ such that for $n \geq n_0$, a reference model within population distance ϵ' of the target exists except with probability at most $\epsilon/8$. Fix such an n . Applying Lemma 1 with confidence $\delta = \epsilon/(8n^2)$ to every pair, a union bound and Lemma 2 give $\max_i \|\hat{\psi}_i - W\psi_i\| \leq c$ except with probability at most $\epsilon/8$, once

$$m \geq C' \frac{d(1+B_R)^2 \bar{B}^2 n^2 \log(n/\epsilon)}{\kappa^4 \lambda_d c^2} = \text{poly}(n, 1/\kappa, 1/\epsilon),$$

for an absolute constant C' (this choice also satisfies the proviso $t_m \leq \kappa/2$ of Lemma 1), and does not involve the pairing fraction ρ . On the intersection of the two events, for a target model f with nearest estimated reference neighbor f^* (perspectives $\psi^*, \hat{\psi}^*$; the target's $\psi, \hat{\psi}$),

$$|\hat{y}(f) - y(f)| \leq \gamma \|\psi^* - \psi\| \leq \gamma (\|\hat{\psi}^* - W\psi^*\| + \|\hat{\psi}^* - \hat{\psi}\| + \|\hat{\psi} - W\psi\|) \leq \gamma(2c + \hat{\delta}),$$

using Assumption 4 and orthogonal invariance. The estimated nearest-neighbor distance $\hat{\delta}$ is at most the estimated distance to the ϵ' -close reference, itself at most $\epsilon' + 2c$ by the same alignment argument, so $|\hat{y} - y| \leq \gamma(4c + \epsilon') \leq \sqrt{\epsilon/2}$. Squaring, and adding the two failure events (squared errors are bounded by 1), $\text{MSE}(\hat{y}) \leq \epsilon/2 + \epsilon/4 \leq \epsilon$. $\square$

Relative to the sample score. The sample score at budget m is the mean of m i.i.d. per-query scores and uses no reference models, so its MSE is σ_q^2/m for every n , where σ_q^2 is the per-query score variance ($\leq 1/4$ for scores in $[0, 1]$). For any $\epsilon < \sigma_q^2/m$ at the budget of Lemma 3, the estimator satisfies $\text{MSE} \leq \epsilon < \sigma_q^2/m$ for all sufficiently large n : query efficiency relative to the sample score in the sense of Definition 1, proving Corollary 1.

Proof of Theorem 2: noisy reference scores. Write the scoring function as $s^{(t)}(f, A) = \frac{1}{|A|} \sum_{q \in A} r(f, q)$ with per-query scores $r \in [0, 1]$, and let each reference cell $Q_i^{(t)}$ be a uniform subsample of $Q^{(t)}$ of size m . Then $\mathbb{E}[\hat{y}_i^{(t)} | f_i] = y_i$ and $\text{Var}(\hat{y}_i^{(t)} | f_i) \leq \sigma_q^2/m$, and the noises $\epsilon_i = \hat{y}_i^{(t)} - y_i$ are independent across i because the subsamples are drawn independently (unpaired designs). For the k -nearest-neighbor regressor, decompose $\hat{y}(f) - y = \frac{1}{k} \sum_{j \in N_k} (y_j - y) + \frac{1}{k} \sum_{j \in N_k} \epsilon_j$ over the neighbor set N_k . The first term is bounded as in the proof of Lemma 3: perspective estimation error (Theorem 1) plus

the population k -th-neighbor distance, which Assumption 5 makes small for n large with $k/n \rightarrow 0$, times the Lipschitz constant. The second term is a mean of k independent, bounded, mean-zero variables, so its conditional variance is at most $\sigma_q^2/(mk)$ and it concentrates by Hoeffding. Choosing $k \rightarrow \infty$ with $k/n \rightarrow 0$ (e.g., $k = \lceil \log n \rceil$) gives $\text{MSE} \leq \varepsilon + \sigma_q^2/(mk) \rightarrow \varepsilon$. Under a common (paired) subsample the ε_i share the draw of the query set and $\text{Cov}(\varepsilon_i, \varepsilon_j)$ equals the variance of the common query-difficulty component, which the neighbor average does not reduce. $\square$

Contrast with paired DKPS. With the exact-matching kernel $k_Q = \delta$ ($k_Q(u, u') = \mathbf{1}\{u = u'\}$), only identical query pairs enter the double sum, so the U -statistic runs over the ρm shared queries and the rate in Lemma 1 becomes $O_P((\rho m)^{-1/2})$: DKPS's guarantee requires $\rho m \rightarrow \infty$ and vacates in the unpaired limit $\rho \rightarrow 0$, where its distance is undefined (Section 3).

Proof of Theorem 2: domination and strict separation. Existence of a dominating bandwidth is witnessed by the exact-matching member: the $k_Q = \delta$ estimator coincides with paired DKPS on the shared queries with the same regression head (Section 3), so $\sigma = \delta$ attains $\text{MSE}(PKPS_\sigma) = \text{MSE}(\text{DKPS})$ at every (n, m) , and any bandwidth improving on exact matching witnesses strict domination. The remainder establishes the operational form: cross-validation attains the best grid member up to ε . Split the reference models into a training set T and a validation set V with $|V| = n_v$. For each $\sigma \in \Sigma$, build the perspective space on T , embed any model outside T out of sample by its Eq. (1) comparison row to T (the standard out-of-sample MDS coordinate), and let $\hat{y}_\sigma$ be nearest-neighbor regression on T 's (perspective, score) pairs. The selected bandwidth is $\hat{\sigma} = \arg \min_{\sigma \in \Sigma} \hat{L}(\sigma)$ with $\hat{L}(\sigma) = \frac{1}{n_v} \sum_{i \in V} (\hat{y}_\sigma(f_i) - y_i)^2$. Conditional on T , the validation losses are i.i.d. across the draw of validation models and bounded in $[0, 1]$, so by Hoeffding's inequality and a union bound over Σ , with probability at least $1 - \delta'$, $\sup_{\sigma \in \Sigma} |\hat{L}(\sigma) - L(\sigma)| \leq \tau = \sqrt{\log(2|\Sigma|/\delta')/2n_v}$, where $L(\sigma)$ is the MSE on a fresh target. On this event,

$$L(\hat{\sigma}) \leq \hat{L}(\hat{\sigma}) + \tau \leq \hat{L}(\delta) + \tau \leq L(\delta) + 2\tau = \text{MSE}(\text{DKPS}) + 2\tau,$$

since the $k_Q = \delta$ member of the family coincides with paired DKPS on the shared queries with the same regression head. Taking n_v large enough that $2\tau \leq \varepsilon$ transfers the domination guarantee to the cross-validated estimator at every (m, ρ) . If some σ^* has $\Delta = L(\delta) - L(\sigma^*) > 0$, the same event gives $L(\hat{\sigma}) \leq L(\sigma^*) + 2\tau = \text{MSE}(\text{DKPS}) - \Delta + 2\tau$, strict for $2\tau < \Delta$. Finally, at $\rho = 0$ with nonatomic designs, $\Pr(q_j = q'_l) = 0$ for every pair of independently drawn queries, so almost surely every identity-matched sum is empty: any measurable completion of the DKPS member predicts as a function of the reference data alone, independent of the target, whence $\text{MSE}(\text{DKPS}) = v + \mathbb{E}[(\mathbb{E}y - \hat{y})^2] \geq v$, while under Assumptions 1–5 the achievability argument supplies a bandwidth attaining $\text{MSE} \leq \varepsilon$, so $\Delta \geq v - \varepsilon$ and the margin is at least $v - \varepsilon - 2\tau \geq v - 2\varepsilon$. In practice we use leave-one-model-out validation and a joint embedding (Section 4); the split protocol is analyzed for independence of the validation losses. $\square$